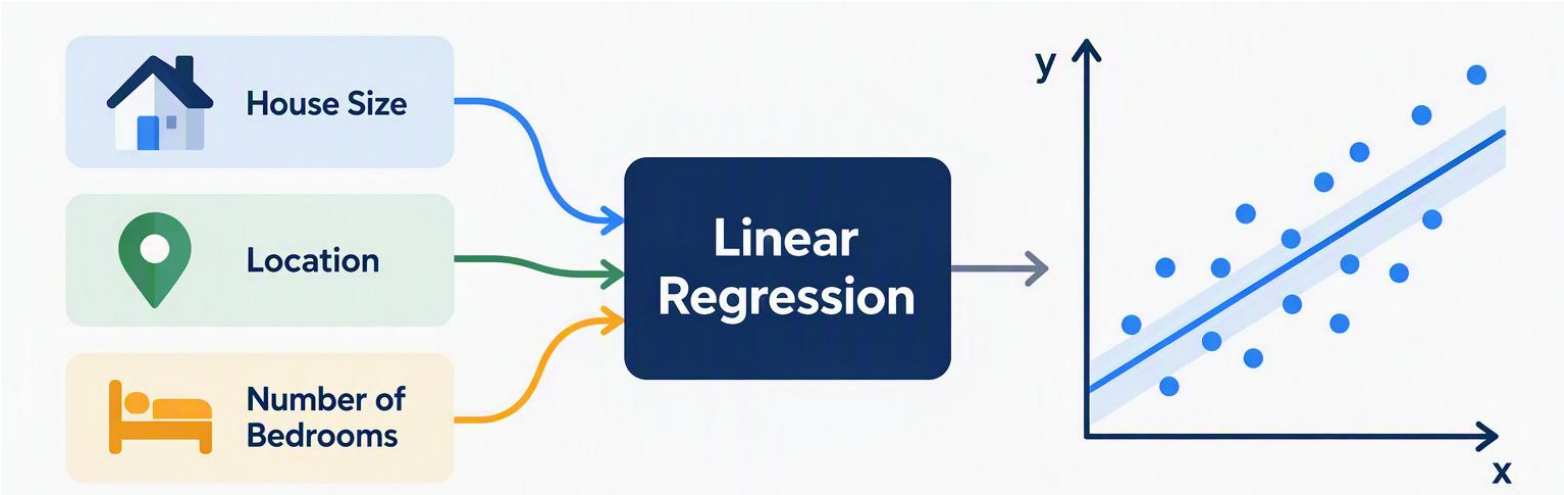
Linear Regression



Dr. Abdelkarim Erradi
CSE@QU

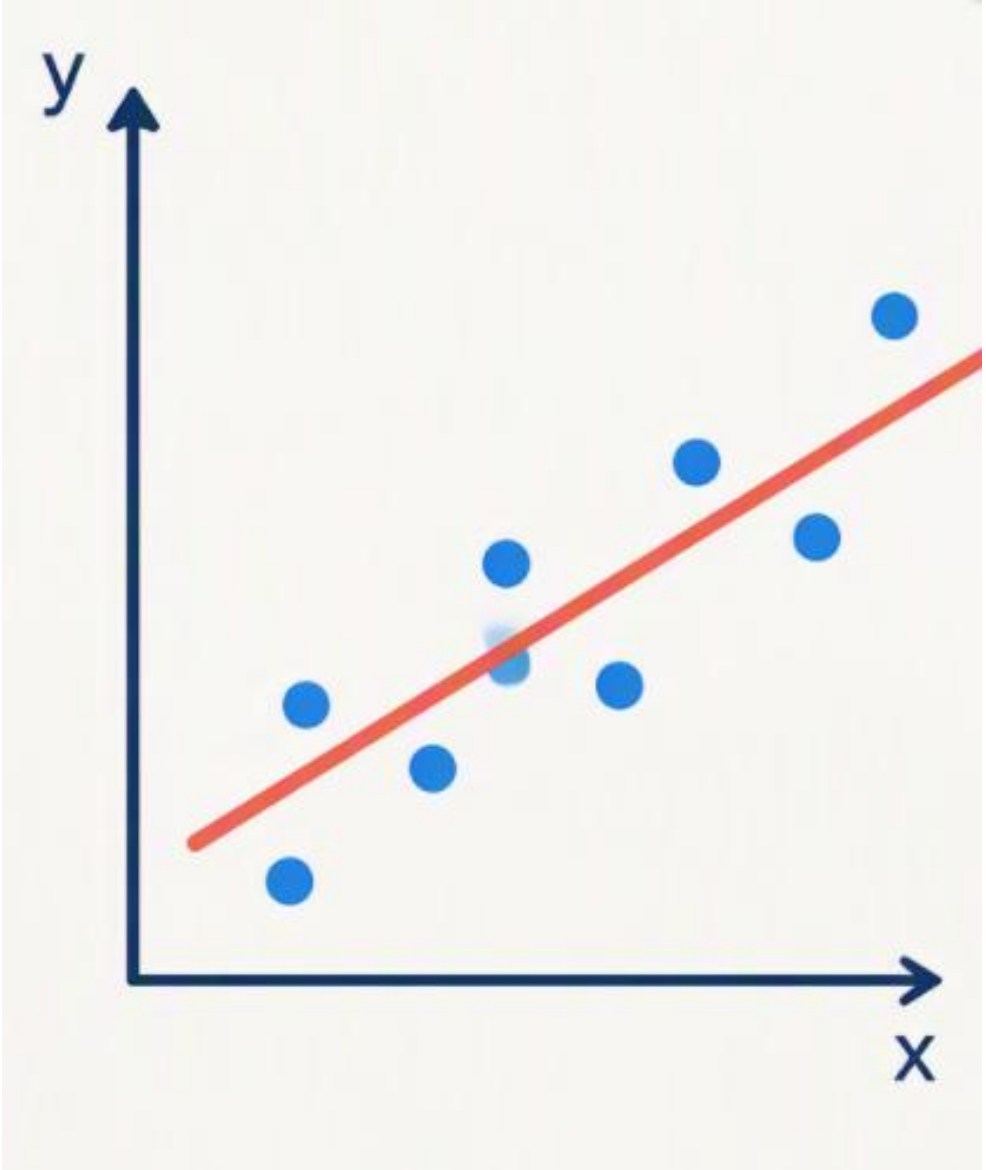
With assistance from AI tools, particularly Microsoft Copilot and Google Gemini

Outline

- Linear Regression with One Variable
- Multiple Linear Regression
- Evaluating and Improving Regression Models
- Model Hyperparameter Tuning

Some of slides are based on the slides from
https://www.deeplearning.ai/specializations/machine-learning

Linear Regression with One Variable



ML: learn a Predictive Function that minimizes the cost

🎯 Objective

Learn a function **f(x)** that predicts the target **ŷ** from input **x** minimizing a **loss function**

0. Initialize: set the model parameters with random values

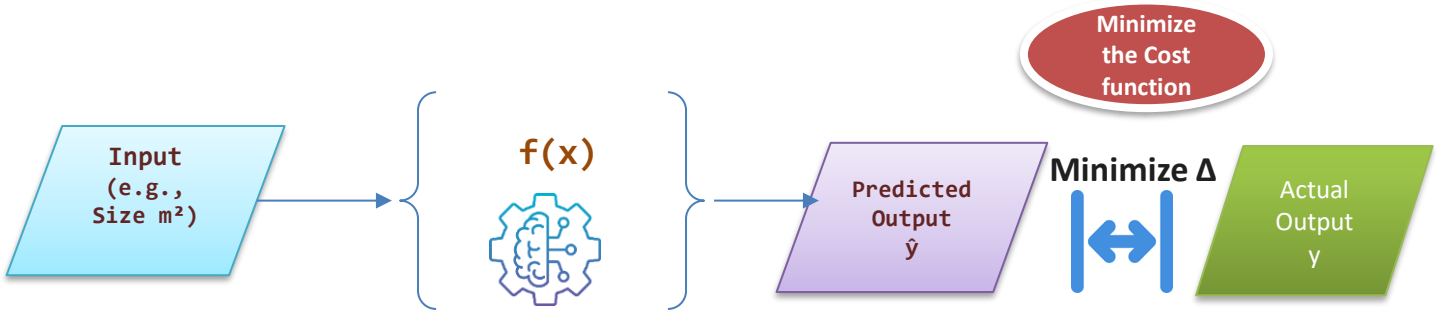
🔁 **Training Loop** (Repeat steps 1–3 until a stopping criterion is met)

1. Predict: compute ŷ = f(x) with current parameters

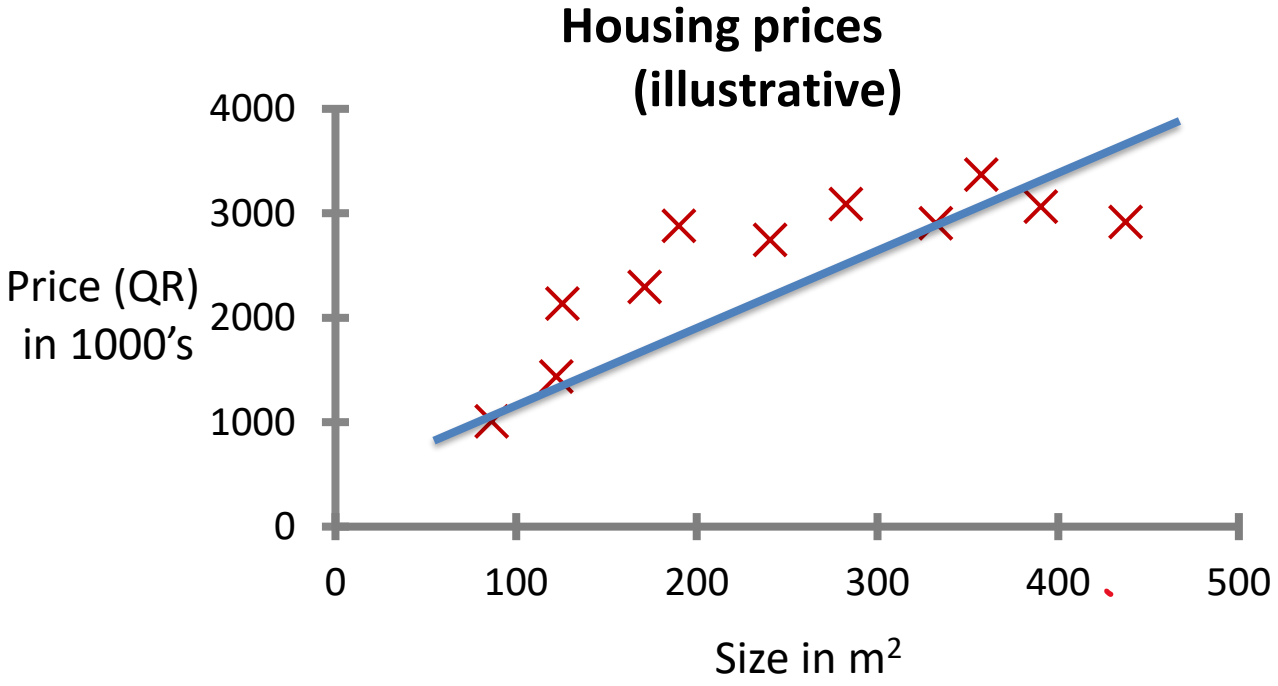
2. Evaluate: Calculate the loss by comparing **ŷ** with the actual **y**

3. Update: Adjust the model parameters to reduce the loss

- Error = Loss = difference between the Predicted Output the Actual Output



Linear Regression with One Variable



Linear regression with one variable aka **Univariate linear regression**

🏠 **Example: Housing Price Prediction**

- **Input (x):** Size of the house (m^2)
- **Output (y):** House price (in QR 1000s)

🧠 **What is Linear Regression?**

- A **supervised learning** algorithm used to predict a **continuous value**
- Predicts output **ŷ** using input **x**
- Fits a **straight line** that best represents the data

Illustrative housing prices

Size in m² (x)	Price (QR) in 1000's (y)
420	4600
280	2320
310	3150
170	1780
…	…

Notation:

m = Number of training examples

x = ***input*** variable (aka feature)

y = ***output*** variable (aka target variable)

$(x^{(i)}, y^{(i)})$ – the ith training example

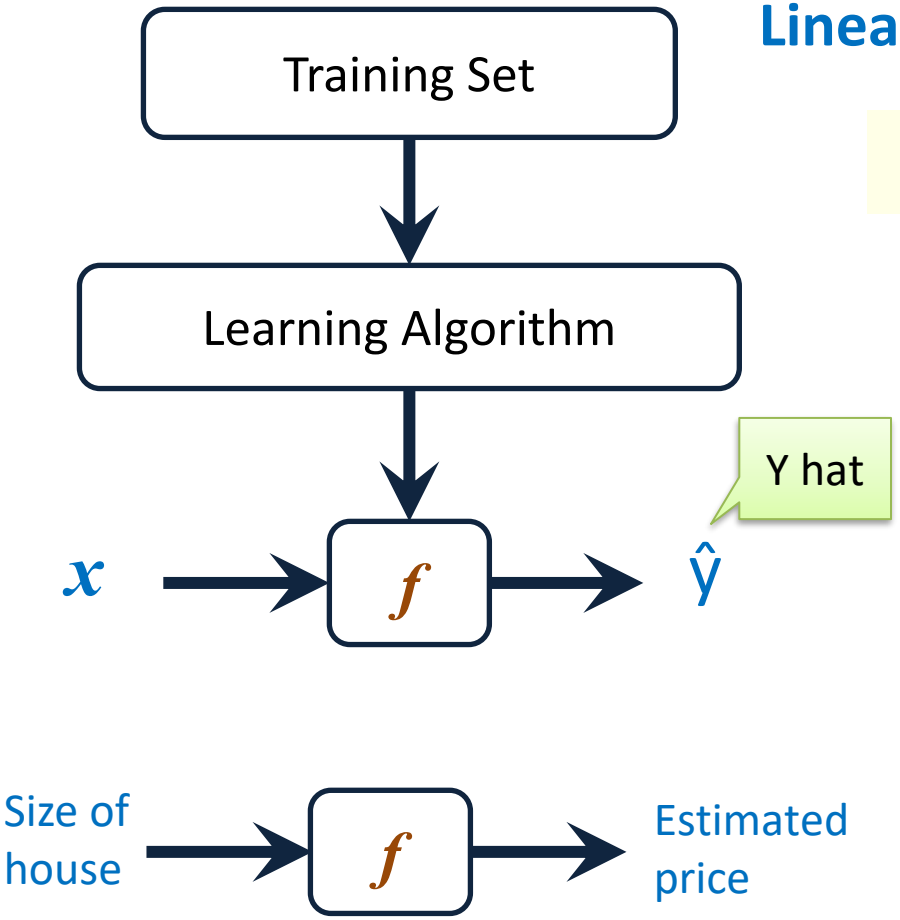
$x^{(1)} = 420$

$y^{(1)} = 4600$

$x^{(2)} = 280$

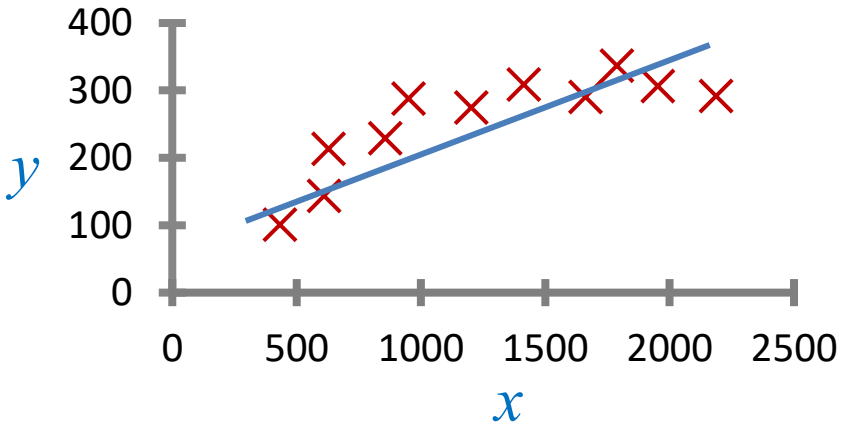
$y^{(2)} = 2320$

Linear Regression Model Representation



$$\hat{y} = f(x) = wx + b$$

w, b are parameters (coefficients) to learn from the training set



🎯 Learning Objective

- Find the **optimal values** w, b that **minimize** the Cost Function:
  - Error between predicted $\hat{y}$ and actual values y
  - **Cost function** = $\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$ over all training examples

Mean Squared Error (MSE)

Model Representation & Parameters

$$f(x) = wx + b$$

- ***w*** is the slope of the line
- ***b*** is the y-intercept of the line

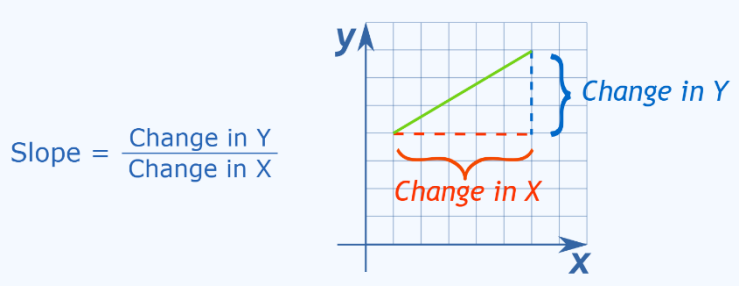
🔍 **How do w and b affect the line?**

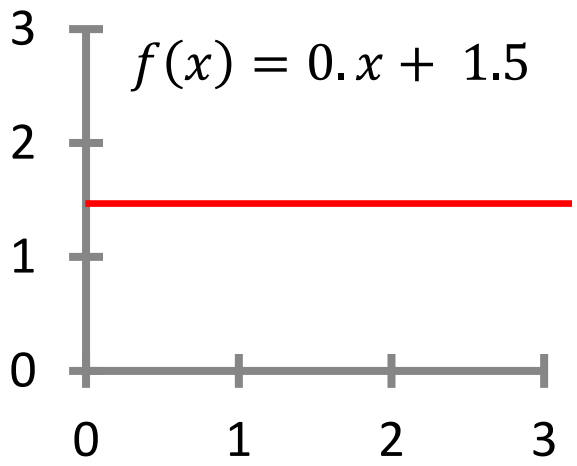
✅ **Changing b** (intercept):

- Moves the line **up or down**
- Controls the predicted value when $x = 0$

✅ **Changing w** (slope):

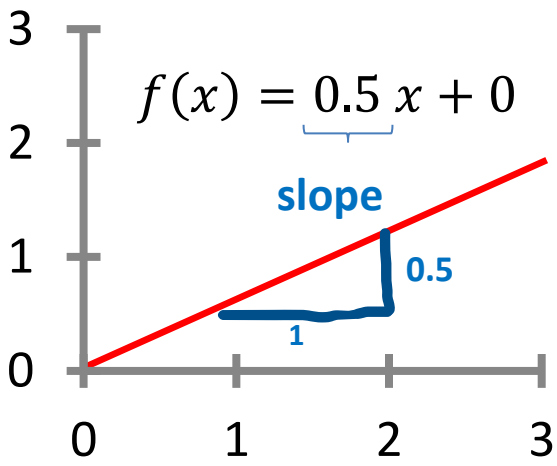
- Changes the **steepness** of the line
- Controls how fast y increases or decreases with x





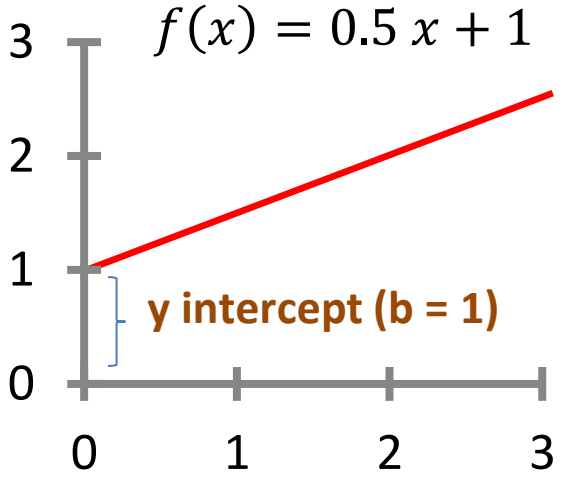
$w = 0$

$b = 1.5$



$w = 0.5$

$b = 0$



$w = 0.5$

$b = 1$



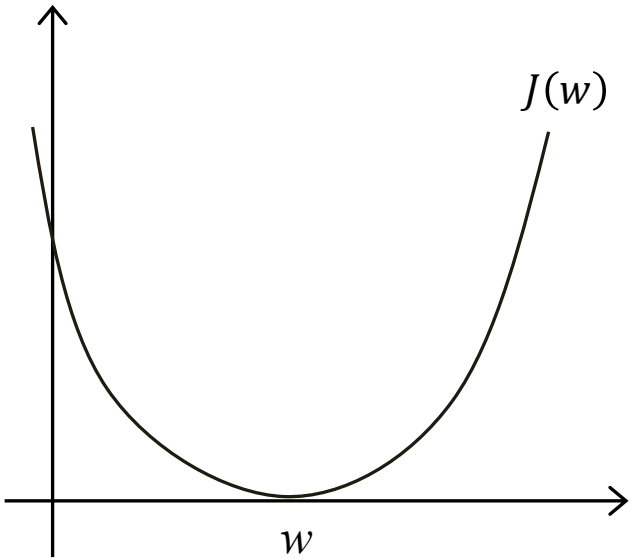
Visualizing the Cost Function $J(w)$

We set **b=0** to allow visualizing J(w) in 2D

$$With\ b=0\ \Rightarrow \boldsymbol{f(x) = wx}$$

🎯 **Learning objective:**
Find the value of w that **minimizes** $J(w)$

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(\mathrm{w}x^{(i)} - y^{(i)})^2$$

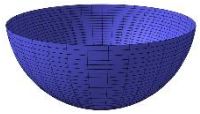


Key Observations

- Each value of w produces:
  - 📈 a different prediction line
  - 📉 a different cost value $J(w)$
- The cost function $J(w)$ has a **bowl-shaped (convex)** curve 🥣
- ✅ Guarantees **one global minimum** that minimizes $J(w)$
- The minimum of $J(w)$ can be efficiently reached by following the slope ➡ this is the basis of Gradient Descent



`slides\regression.xlsx`



Visualizing the Cost Function $J(w,b)$

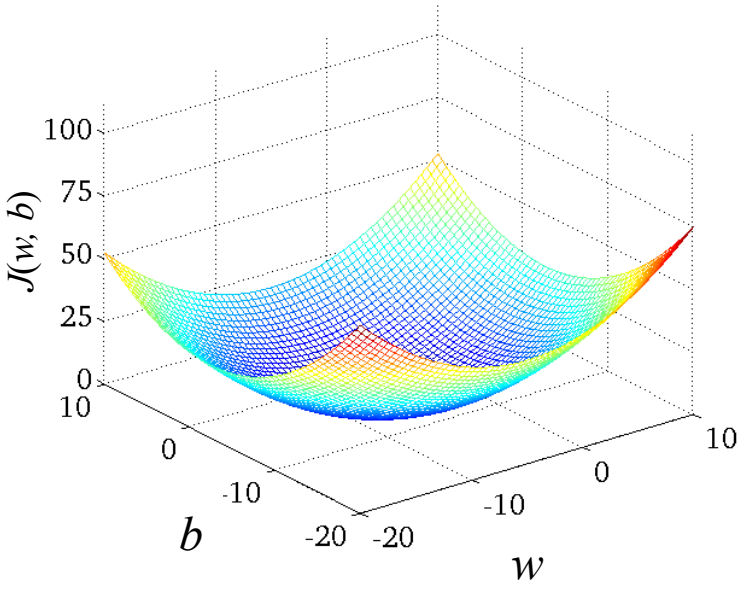
$$f(x) = wx + b$$

Linear Regression Model

$$f(x) = wx + b$$

Cost Function (MSE):

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(f(x^{(i)}) - y^{(i)})^2$$

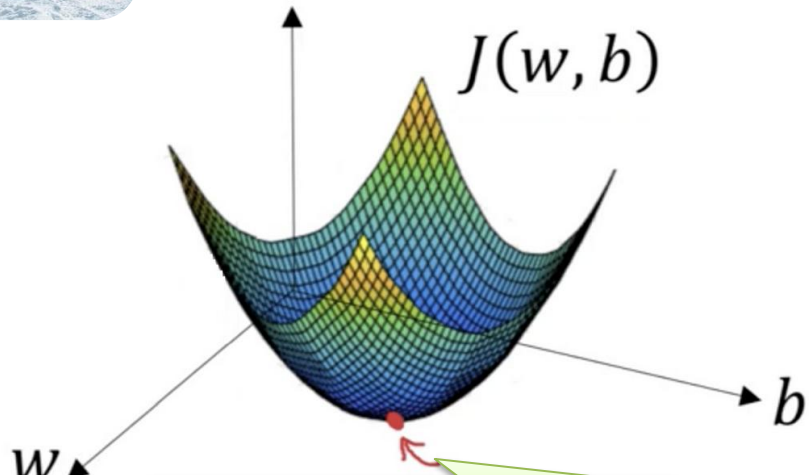


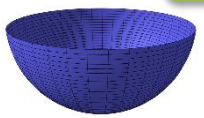
Key Observations 🔍

- Each pair (w, b)produces:
  - 📈 a different prediction line
  - 📊 a different cost value
- When **both** w **and** b are considered:
  - $J(w, b)$forms a **convex, bowl-shaped surface** 🎨
  - ✅ Only **one global minimum** exists
- The minimum lies at the **bottom of the bowl** ⬇️

➡️ This minimum can be efficiently reached by **following the slope** of the surface → Gradient Descent







Bowl shape
Convex function

$J(w,b)$

🧠 Key Idea

Gradient Descent iteratively **moves downhill** on the cost surface to **find the optimal w** and **b** that minimizes that cost function

➡️ This is the core optimization method used to train linear regression models

Gradient Descent Algorithm

Goal 🎯: Learn the optimal parameters w and b that minimize the cost function.

Univariate Linear Regression Model:

$$\hat{y}^{(i)} = f\left(x^{(i)}\right) = wx^{(i)} + b$$

Cost Function (Mean Squared Error):

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$$

Where:

- **m** = number of training examples
- i = index of a training example

$$i = 1,2,\dots,m$$

- $x^{(i)}$ = input feature of example i
- $y^{(i)}$ = actual target value of example i
- $\hat{y}^{(i)}$ = predicted value of example i

1. **Initialize parameters with random values** (e.g., $w = 0,\ b = 0$)
2. **Compute Gradients (Partial Derivatives)** 📐

Measure how the cost changes with respect to each parameter:

$$\frac{\partial J}{\partial w} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})\,x^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})$$



07.regression\
04_gradient_descent_one_var

3. **Update parameters** 🔄

Updates move parameters **downhill** ⬇️ toward lower cost

α:learning rate 🎚️

$$w := w - \alpha\frac{\partial J}{\partial w}$$

$$b := b - \alpha\frac{\partial J}{\partial b}$$

4. **Repeat** 🔁 2 and 3 until the cost stops decreasing (convergence ✅)

Derivatives: Meaning and Use

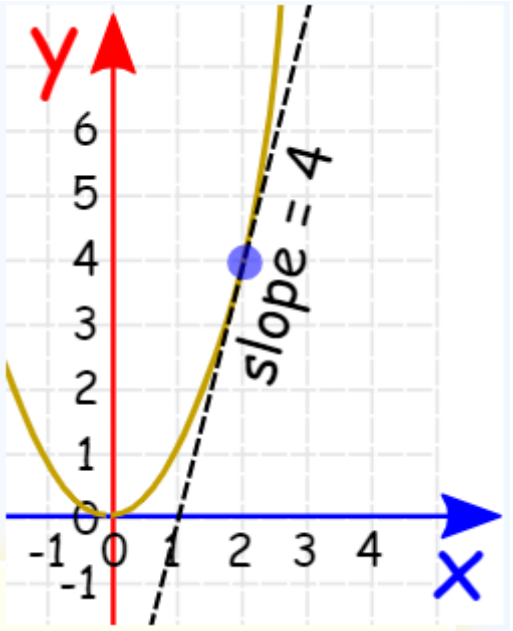
Meaning

- The derivative is the slope at a point: how fast **y** changes for a small change in **x**.

- The derivative is the tangent slope at one point.

Example: $f(x) = x^2$, so $f'(x) = 2x$

At x = 2: slope = 4.
At x = 5: slope = 10, so the curve rises more steeply.



🧠 Role in gradient descent

➡️ Derivatives give us the **cost function's slope**

➡️ Update parameters in the **opposite** direction to reduce cost.



07.regression\
gradient_decent_x_square_animation.py

The Impact of the Learning Rate (α)

Gradient Descent Update Rule $$w \leftarrow w - \alpha \frac{dJ(w)}{dw}$$

The learning rate **α controls the step size** taken in the direction of the negative gradient.

If α is too small

Updates to w are **very small** → $J(w)$ decreases slowly

✅ Guaranteed to move toward the minimum → slow but safe

❌ **Convergence is slow:** *takes many iterations*

If α is too large

Updates to w are **too big** → fast but risky

❌ The algorithm may **overshoot the minimum:** $J(w)$ may oscillate or even increase

❌ **May fail to converge :** *Gradient descent becomes unstable*

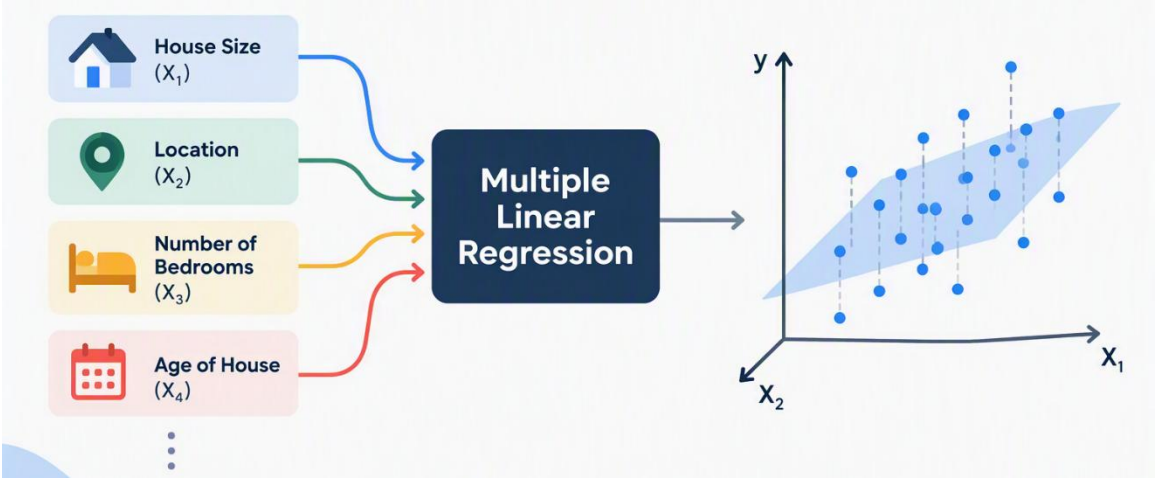
✅ Choosing α in Practice

- Start with a **moderate value** (e.g., 0.01 or 0.1)
- Select an α where $J(w)$ decreases smoothly on every iteration

👍 **Good α → fast and stable convergence**

→ If gradient descent is not working, first try reducing α before changing anything else

Multiple Linear Regression



Linear Regression with multiple variables

Illustrative data

Size (m^2)	Number of bedrooms	Number of floors	Age of home (years)	Price (QR 1000)
420	5	1	45	4600
280	3	2	40	2320
310	3	2	30	3150
170	2	1	36	1780
…	…	…	…	…

Notation:

n = number of features

x_j = j^{th} feature

$\vec{x}^{(i)}$ = features of ith training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

Multiple Linear Regression:

$$f(\boldsymbol{x}) = \boldsymbol{w_1 x_1} + \boldsymbol{w_2 x_2} + \cdots + \boldsymbol{w_n x_n} + \boldsymbol{b}$$

Example

$f(x) = 0.1\,x_1 + 4\,x_2 + 10\,x_3 + -2\,x_4 + 80$

x_1: size; x_2: #bedrooms; x_3: #floors; x_4: years; 80: base price

📐 Multiple Linear Regression — Vector Form

📊 Model Representation

Given **m** training samples and **n** input features, the **vectorized linear model** is

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

Where:

$X \in \mathbb{R}^{m \times n}$ input dataset matrix,
$m =$ number of samples,
$n =$ number of input features,
$\mathbf{w} \in \mathbb{R}^{n}$ (n)−dimensional weight vector,
$b \in \mathbb{R}$ bias term (scalar),
$\hat{\mathbf{y}} \in \mathbb{R}^{m}$ (m)−dimensional output vector

X dataset matrix	w parameters vector
$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$	$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$

Rows → samples
Columns → features

🎯 Learning Objective

Learn parameters $\mathbf{w}$ and b by minimizing the **Mean Squared Error (MSE)**:

$$J(\mathbf{w}, b) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

🔁 **Gradient Descent (Vectorized)**. All parameters are updated **simultaneously**:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\frac{\partial J}{\partial \mathbf{w}}$$

$$b \leftarrow b - \alpha\frac{\partial J}{\partial b}$$

🚀 Key Takeaways

- Multiple Linear Regression expressed in vector form enables efficient computation, clean notation, and scalable learning
- Same gradient descent algorithm — just applied to vectors and matrices 💡

📐 Gradient Descent (Vectorized)

🔁 Gradient of the MSE cost with respect to the weights w:

$$dw = \frac{\partial J}{\partial w} = \frac{2}{m} X^T (\hat{y} - y)$$

i = 1, …, m sample index (row of X)

j = 1, …, n feature index (column of X)

Why X^T? Because it makes the shapes work out:

X^T (n × m)	×	$\hat{y} - y$ (m × 1)	=	**dw** (n × 1)
n features × m samples		one error per sample, i = 1…m		one gradient per weight, j = 1…n

- **For each weight** $j = 1…n$: $dw_j = (2/m) \cdot \Sigma_{i=1…m}\, x_{ij}\, (\hat{y}_i - y_i)$
- x_{ij} = value of feature j for sample i (row i, column j of X)
- **Bias:** $db = (2/m) \cdot \Sigma_{i=1…m}\, (\hat{y}_i - y_i)$
- **Update:** $w \leftarrow w - \alpha \cdot dw, \quad b \leftarrow b - \alpha \cdot db$

Worked Example: X^T and the Update of w

① Compute $\hat{y} = Xw + b$ then the error $e = \hat{y} - y$ (m = 4 samples, n = 2 features)

$$\underset{X\ (4\times2)}{\begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 3 & 0 \\ 4 & 1 \end{bmatrix}} \times \underset{w\ (2\times1)}{\begin{bmatrix} 1 \\ 1 \end{bmatrix}} + \underset{b}{\begin{bmatrix} 0 \end{bmatrix}} = \underset{\hat{y}}{\begin{bmatrix} 3 \\ 3 \\ 3 \\ 5 \end{bmatrix}} - \underset{y\ (true)}{\begin{bmatrix} 4 \\ 2 \\ 6 \\ 5 \end{bmatrix}} = \underset{e\ (4\times1)}{\begin{bmatrix} -1 \\ 1 \\ -3 \\ 0 \end{bmatrix}}$$

② Transpose $X^T e$ sums each feature's column against the errors

$$\underset{X^T\ (2\times4)}{\begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 0 & 1 \end{bmatrix}} \times \underset{e\ (4\times1)}{\begin{bmatrix} -1 \\ 1 \\ -3 \\ 0 \end{bmatrix}} = \underset{X^Te\ (2\times1)}{\begin{bmatrix} -8 \\ -1 \end{bmatrix}}$$

$dw = (2/4) \cdot [-8, -1] = [-4, -0.5]$

Row j of X^T = column j of X:

$-8 = 1(-1) + 2(1) + 3(-3) + 4(0)$

$-1 = 2(-1) + 1(1) + 0(-3) + 1(0)$

③ Update $w \leftarrow w - \alpha \cdot dw$ with learning rate $\alpha = 0.1$

$$w \leftarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} - 0.1 \times \begin{bmatrix} -4 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 1.4 \\ 1.05 \end{bmatrix}$$

$db = (2/4)(-1+1-3+0) = -1.5$

$b \leftarrow 0 - 0.1 \cdot (-1.5) = 0.15$

Both gradients are negative, so w and b increase and $\hat{y}$ moves up toward y (the model was under-predicting on average). Repeat until J stops decreasing.

House-price data: area, rooms, age

Goal: predict a house's price (QR) from three features: its area (m^2), number of rooms and age (years).

#	Area (m^2)	Rooms	Age (yrs)	Price (QR)
1	80	3	20	448,000
2	95	2	8	510,000
3	110	4	25	587,000
4	125	3	12	654,000
5	140	5	5	840,000
6	155	3	18	738,000
7	170	4	10	899,000
8	185	6	3	1,061,000
9	200	4	15	992,000
10	220	5	6	1,141,000

Three features, three scales

Area 80 – 220 m^2

spread: 140

Rooms 2 – 6

spread: 4

Age 3 – 25 years

spread: 22

Raw numbers differ by up to 35×. Feature scaling is needed (see slides 22–23).

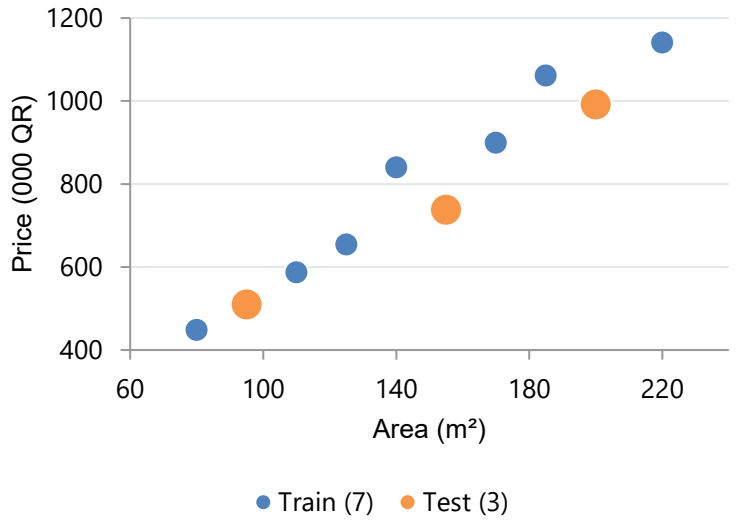
```
X = df[["area", "rooms", "age"]]   # features: 10 houses × 3 columns
y = df["price"]                    # target: one known price per house
```

X shape (10, 3): 10 houses (m) × 3 features (n) **y shape (10,): one price per house**

1. Train/test split

Split first: the test houses must stay unseen.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```



Train: 7 houses → fit

Areas: 80, 110, 125, 140, 170, 185, 220 m^2

X_train, y_train:

→ Fit the **scaler** on X_train

→ Fit the **model** on the scaled training data

Test: 3 houses → score

Areas: 95, 155, 200 m^2 (510,000 / 738,000 / 992,000 QR)

X_test, y_test: hidden until scoring

test_size=0.3 → 3 of 10 houses; random_state=42 → same split every run.
Only 10 rows: fine for learning, use far more data in practice.

Defining Training and Test Set Splits



📘 **Training Set (70–80%)**:

- Used to **learn patterns** and adjust model parameters

🧪 **Test Set (20–30%)**

- Never used in training
- Measures true performance on unseen data

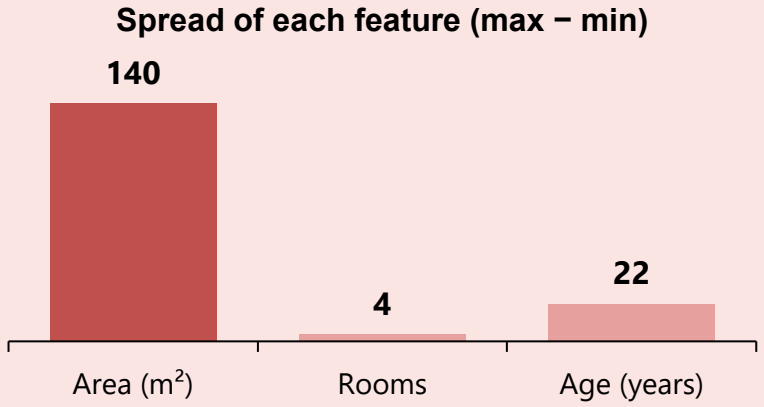
🎯 **Why Split the Data?**

- Avoid **data leakage**
- Get an **unbiased** model evaluation
- Detect **overfitting vs. underfitting**
- Enable fair model comparison

Why scale features?

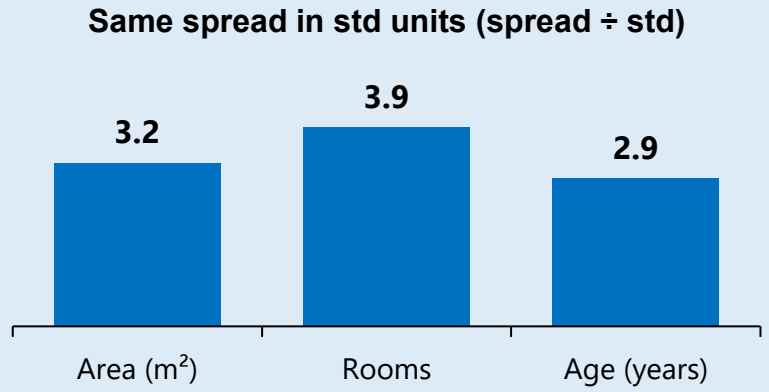
Big numbers can dominate the model just because of their units.

✗ Without scaling: area dominates



Area has much larger numerical values (35× bigger) than Rooms or Age, so it looks "more important" only because of its unit.

✓ With scaling: fair comparison



All features are now expressed in standard-deviation units, making their numerical scales comparable.

How: z = (x − mean) ÷ std every feature gets mean 0 and std 1

Example: area 80 m^2 → (80 − 147.1) ÷ 44.2 = −1.52 (1.52 std below average)

Scaling does not make features equally important. It makes their numerical scales comparable.

Why scaling matters: Puts features on a similar scale, so large-valued features do not dominate. Especially important for Gradient Descent.

2. Standardize the features

StandardScaler: z = (x − mean) / std, one mean and std per feature

Area (m^2)	z_area	Rooms	z_rooms	Age (yrs)	z_age
80	−1.52	3	−1.25	20	1.11
185	0.86	6	1.66	3	−1.13
110	−0.84	4	−0.28	25	1.77
220	1.65	5	0.69	6	−0.74
140	−0.16	5	0.69	5	−0.87
125	−0.50	3	−1.25	12	0.06
170	0.52	4	−0.28	10	−0.21
mean 147.1	**0.00**	**mean 4.29**	**0.00**	**mean 11.57**	**0.00**
std 44.2	**1.00**	**std 1.03**	**1.00**	**std 7.58**	**1.00**

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # learns mean, std on TRAIN
X_test_s  = scaler.transform(X_test)        # reuses the same mean, std
```

Example: area 80 m^2

z_area = −1.52

(80 − 147.1) / 44.2

Reading a z-score

0 = average

+1 / −1: one std above / below the train mean

Never fit on test data

No leakage

Test houses are scaled with train stats

3. Model training and prediction

`model.fit()` learns w and b on the scaled training data

`model.predict()` needs X_test scaled with the same scaler

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train_s, y_train)     # learn w and b on scaled data
print(model.coef_)                # [173003.  44472. -33619.]  per +1 std
print(model.intercept_)           # 804286.  = b
y_pred = model.predict(X_test_s)  # one price per test house
```

price ≈ 804,286 + 173,003·z_area + 44,472·z_rooms − 33,619·z_age

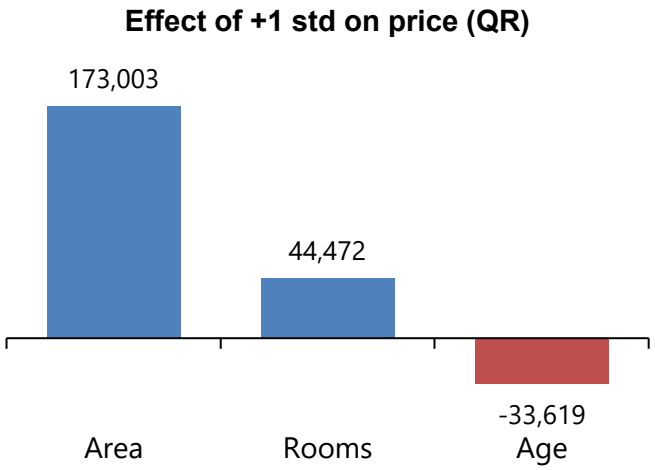
- **Intercept b** = 804,286 QR: predicted price when every feature equals its training-set average (avgs: 147 m² area, 4.3 rooms, 11.6 years)

- Coefficients w = predicted price change for a one-standard-deviation increase, holding other features fixed.

- Converting weights back to real units:

$$\text{effect per original unit} = \frac{\text{scaled coefficient}}{\text{feature standard deviation}}$$

Area: $173{,}003/44.2 \approx 3{,}914$QR per m²
Rooms: $44{,}472/1.03 \approx 43{,}177$QR per room
Age: $-33{,}619/7.58 \approx -4{,}435$QR per year



4. Evaluate the Model

Compare predictions with the known test prices

Test house	Actual (QR)	Predicted (QR)	Error (pred − actual)
200 m^2 · 4 rooms · 15 yrs	992,000	983,492	−8,508
95 m^2 · 2 rooms · 8 yrs	510,000	517,499	+7,499
155 m^2 · 3 rooms · 18 yrs	738,000	750,986	+12,986

```
from sklearn.metrics import (mean_absolute_error,
        mean_squared_error, r2_score)
mae  = mean_absolute_error(y_test, y_pred)          # 9664
rmse = mean_squared_error(y_test, y_pred) ** 0.5    # ≈ 9954
r2   = r2_score(y_test, y_pred)                     # 0.997
```

MAE

9,664 QR

average miss: (8,508 + 7,499 + 12,986) / 3

RMSE

9,954 QR

like MAE, but punishes big misses more

R^2

0.997

% of price variation explained by the model

- MAE ≈ 1.2% of the average training price (804,286 QR), so errors are small.
- Judge error in the units of y

4. Predict the price of a new house

New house: 150 m², 4 rooms, 10 years old

```
new = [[150, 4, 10]]              # area, rooms, age (raw units)
new_s = scaler.transform(new)     # same mean/std as training
price = model.predict(new_s)[0]
print(round(price))               # 810101
```

≈ 810,101 QR z-scores: area 0.065, rooms −0.277, age −0.207

= 804,286 + 173,003 × 0.065 + 44,472 × (−0.277) − 33,619 × (−0.207)

Typical error ≈ ±10,000 QR (test MAE): roughly 800,000 – 820,000 QR.

Before you trust it

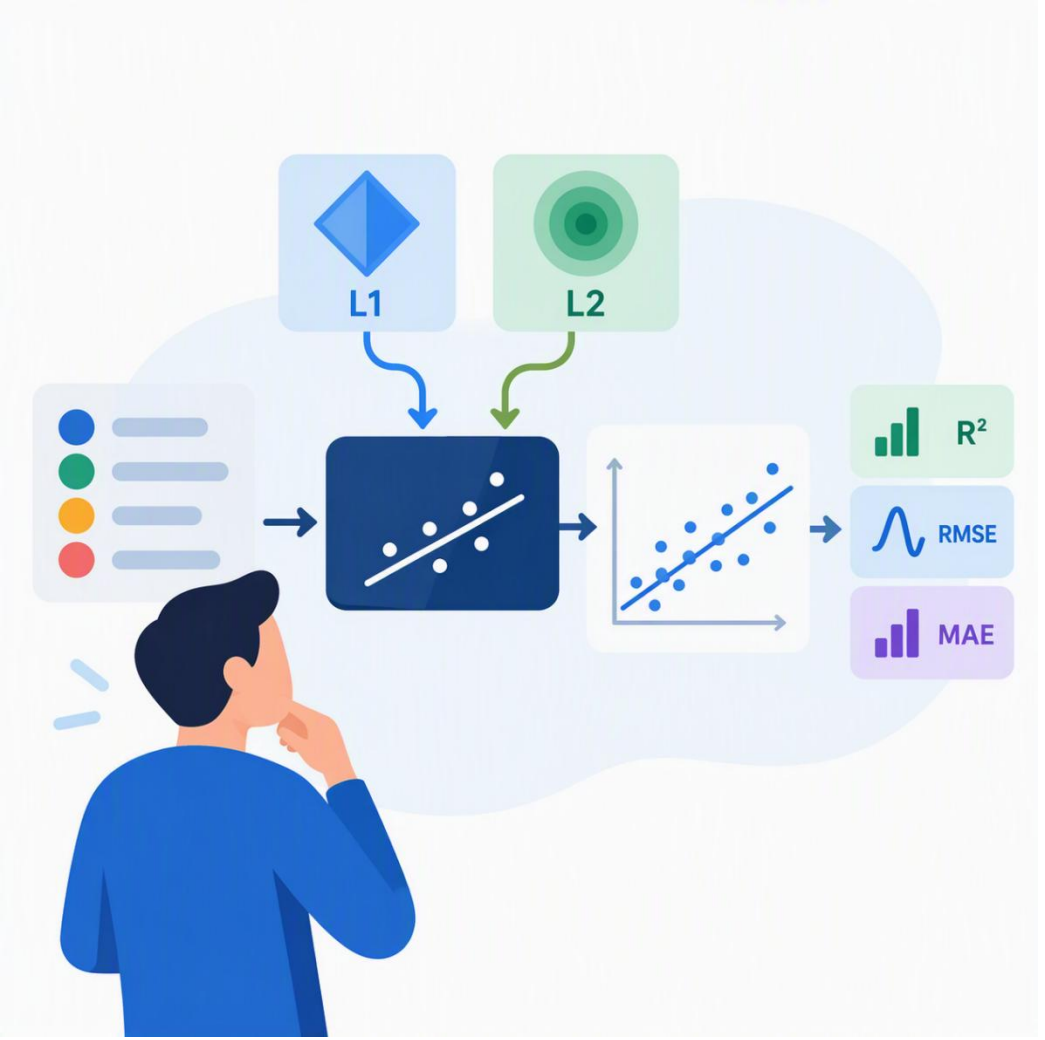
Stay in range

Trained on 80–220 m², 3–6 rooms, 3–25 years: this house is inside all three. A 400 m² villa would be extrapolation.

Scale every new input

Always transform, never refit. Raw [150, 4, 10] fed to this model would predict ≈ 26.6 million QR!

Evaluating and Improving Regression Models



Regression Error Metrics

MSE
Mean squared error

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

Units: QR² (squared)

Best for training
Squares each error, so big misses dominate. Use it to compare models, not to report.

RMSE
Root mean squared error

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2}$$

Units: QR (same as y)

Best when big misses are costly
Typical error in QR, extra weight on big misses.
Square root brings MSE back to y's units.

MAE
Mean absolute error

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

Units: QR (same as y)

Best for reporting
Average miss in QR; all errors count equally.
Easiest to explain; robust to outliers.

Reading the house-price test together

RMSE (12,665) > MAE (11,800): errors are uneven (7,200 vs 16,400). A big gap flags outliers to inspect.

Always RMSE ≥ MAE. Lower is better; 0 means perfect predictions.

Is that good enough? Compare with a baseline and a tolerance (next slide).

When Is a Regression Model Good?

Good = beats average training price and is accurate enough for the task.

1
Beat the baseline

Baseline = average training price (830,000 QR)
Predict 830,000 QR for every test house, then calculate the prediction errors.

Test house	Actual price (QR)	Baseline guess (QR)	Error = gap (QR)
100 m^2	610,000	830,000	**220,000**
180 m^2	960,000	830,000	**130,000**

Baseline MAE = (220,000 + 130,000) ÷ 2 = 175,000 QR

Our model's MAE is only 11,800 QR: about 93% lower. It clearly beats the baseline.

2
Small enough?

Ask: how much error can the task accept?
Our MAE of 11,800 QR is about 1.4% of the baseline price (830,000 QR)
Whether this is good depends on the application:
Acceptable average error = ±20,000 QR → ✅ Acceptable
Acceptable average error = ±10,000 QR → ❌ Needs improvement

3
Check mistakes

Are the errors random or one-sided?
Both errors are negative (predicted − actual): −7,200 and −16,400 QR. The model guessed too low for both houses.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

R^2 (R-squared)

1 The idea

R^2 compares your model with the mean price baseline

R^2 = 1 − (model's squared error ÷ baseline's squared error)

2 House example

Baseline = mean test price. Errors^2 in millions of QR^2.

House	Actual (QR)	Baseline y	Baseline error^2	Model $\hat{y}$	Model error^2
100 m^2	610,000	785,000	30,625	602,800	51.84
180 m^2	960,000	785,000	30,625	943,600	268.96
Total			**61,250**		**320.8**

R^2 = 1 − 320.8 ÷ 61,250 = 0.995

Model error is only 0.5% of the baseline error.

3 How to read it

R^2 = 1
Perfect predictions

R^2 = 0
No better than the baseline

R^2 < 0
Worse than the baseline

R^2 is not accuracy

R^2 = 0.995, yet the model still misses by 11,800 QR on average.

Report MAE or RMSE too, and compute all of them on test data.

Underfitting vs Overfitting

Compare training error with test error to diagnose the fit.

Underfit Too simple (high bias)	Good fit Generalizes well	Overfit Too complex (high variance)
Training error: HIGH **Test error: HIGH** Misses the pattern, like a straight line through curved data. Example: train RMSE 90k, test RMSE 95k QR	**Training error: LOW** **Test error: LOW** Captures the real pattern and ignores noise. Train and test errors are close. Example: train RMSE 12k, test RMSE 14k QR	**Training error: VERY LOW** **Test error: HIGH** Memorizes noise, like a wiggly curve through every point. Example: train RMSE 2k, test RMSE 60k QR
What to do • Add relevant features • Try polynomial regression • Reduce regularization	**What to do** • Keep it • Re-check on fresh data • Retrain when data changes	**What to do** • Apply Ridge / Lasso • Add more training data • Drop noisy features

Practical rule

Large gap between train and test error = overfit. Both high = underfit. Tune on validation data and keep the test set for the final check.

Regularization: Motivation & Idea

Regularization = Penalize large weights to reduce overfitting

- **Problem:** a complex model may learn **noise instead of the true pattern**. Its weights can become large, making predictions overly sensitive: a small change in x may cause a large change in $\hat{y}$ → **overfitting** ⚠️
- **Idea:** add a penalty to the cost function to **discourage large weights**, encouraging a simpler, less sensitive model.

New cost = MSE + **λ** × penalty on weights

Strength λ: λ determines how much we penalize large weights. Start with λ = 1. If the model still overfits, increase λ; if it underfits, decrease λ.

- **Effect:** weights shrink. Training error rises slightly, but error on new data drops.

```
from sklearn.linear_model import LinearRegression, Ridge

model = LinearRegression()   # no penalty: may overfit
model = Ridge(alpha=1.0)     # penalty on weights; alpha = λ
```

Regularization Example

Same house data, three models. Compare the learned weights.

Feature	No regularization	Ridge (L2)	Lasso (L1)
Size	1200	850	900
Rooms	8000	6000	6200
Age	−150	−120	−130
Noise level	**−30000**	−8000	**0**
Park score	**95000**	15000	**0**

No regularization
Large weights on noisy features → Overfits.

Ridge
Use every feature, but don't trust any one too much.

Lasso
If a feature doesn't help enough, remove it (weight = 0).

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso

linear = LinearRegression().fit(X_train, y_train)
ridge  = Ridge(alpha=1).fit(X_train, y_train)
lasso  = Lasso(alpha=1).fit(X_train, y_train)

print("Linear:", linear.coef_)
print("Ridge: ", ridge.coef_)
print("Lasso: ", lasso.coef_)
```

Ridge Regularization (L2)

Penalty = sum of squared weights. It shrinks every weight.

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 + \frac{\lambda}{m}\sum_{j=1}^{n} w_j^2$$

MSE term
How wrong the predictions are on the training data.

Penalty term: $\frac{\lambda}{m}\sum_{j=1}^{n} w_j^2$
The larger the weight, the larger the penalty → encourages smaller weights.

λ (lambda): Penalty strength
Small λ → less weight shrinkage
large λ → more weight shrinkage
Choose λ to balance overfitting vs. underfitting.

Effect of L2 Regularization

✅ Keeps all features in the model

✅ Shrinks large weights toward zero → reduces sensitivity to small changes in the inputs

✅ Can reduce overfitting and improve generalization

❌ Does not eliminate features because weights rarely become exactly zero

```
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0).fit(X_train, y_train)   # alpha plays the role of λ
print(model.coef_)   # small, non-zero weights
```

Lasso Regularization (L1)

Penalty = sum of absolute weights. It pushes weak weights to 0.

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 + \frac{\lambda}{m}\sum_{j=1}^{n}|w_j|$$

MSE term
How wrong the predictions are on the training data.

Penalty term: $\frac{\lambda}{m}\sum_{j=1}^{n}|w_j|$
Every non-zero weight adds to the penalty, this pushes weak weights to exactly 0 → automatic feature selection.

λ (alpha)
Penalty strength. Small = weak, large = strong. Tune it.

Effect of L1 Regularization

- ✅ Sets some weights exactly to 0
- ✅ Removes weak or less useful features → **automatic feature selection**
- ✅ Produces a simpler, easier-to-interpret model

Example: If Noise and Park Score get weight = 0, the model effectively uses only Size, Rooms, and Age.

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=1.0).fit(X_train, y_train)
print(model.coef_)   # zeros = features dropped
```

L1 vs L2 Comparison

Both shrink weights 

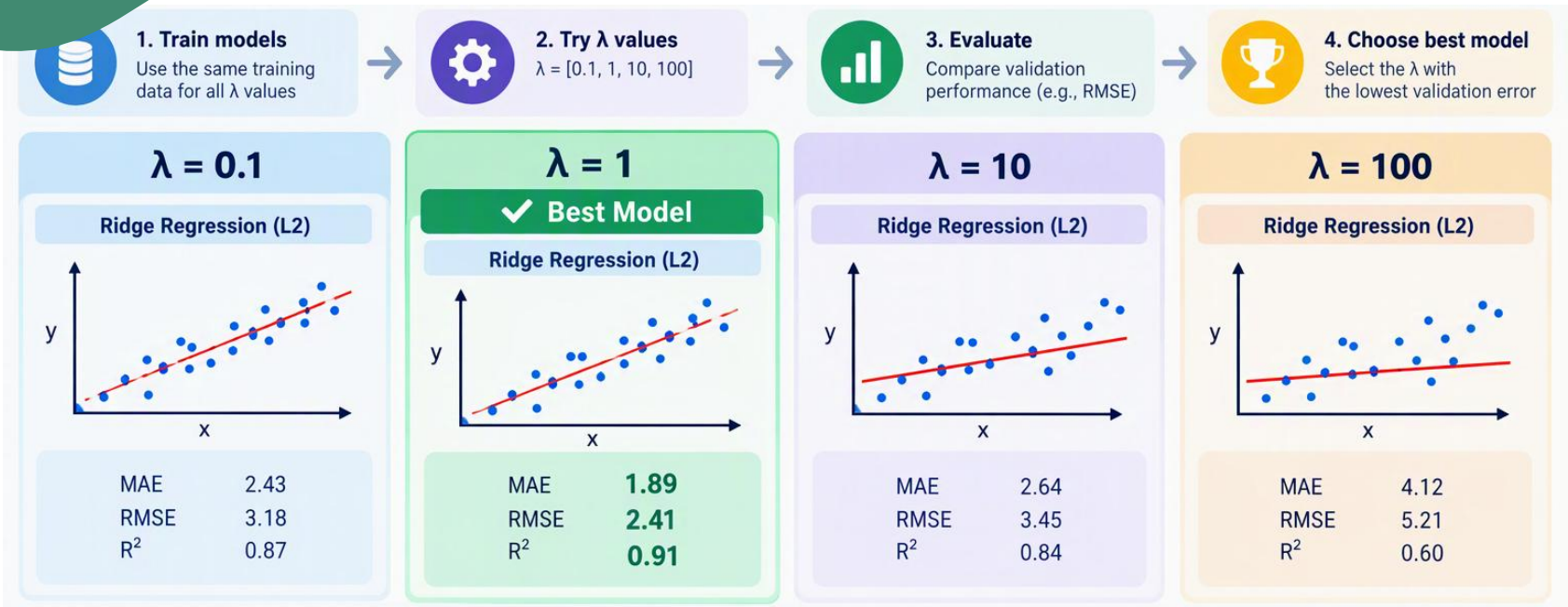
Aspect	Ridge (L2)	Lasso (L1)
Penalty	Sum of squared weights	Sum of absolute weights
Weights	Shrink smoothly, rarely reach 0	Weak ones become exactly 0
Features	Keeps all features	Keeps a subset (selection)
Best when	Many useful, related features	Many noisy or useless features
Stability	High	Medium
scikit-learn	`Ridge(alpha=1.0)`	`Lasso(alpha=1.0)`

Which one?
Start with Ridge. Try Lasso when you suspect many useless features. Keep the one with the better test score.

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=1.0).fit(X_train, y_train)
print(ridge.score(X_test, y_test), lasso.score(X_test, y_test))   # R²
```

Model Hyperparameter Tuning



🎛️ Model Hyperparameter Tuning

Hyperparameter tuning = try candidate settings, score each with cross-validation (CV), and keep the best one.

	Parameters	Hyperparameters
Set by	Learned from data during training	Set by the ML developer before model training
Examples	Weights w and bias b	λ (alpha), learning rate α, folds K
Found by	Gradient descent / .fit()	Grid search + cross-validation

🔄 The tuning process

🧩 **1 Define Search Space**

Choose candidate values for each hyperparameter e.g., λ ∈ {0.01, 0.1, 1, 10, 100}

➡️

🏋️ **2 Train**

Train the model on K−1 folds for each candidate setting

➡️

📏 **3 Validate**

Evaluate on the held-out fold and average the scores across K folds

➡️

🏆 **4 Select Best**

Choose the hyperparameter setting with the best average CV score

➡️

✅ **5 Refit and Evaluate**

Retrain on the full training set and evaluate on the test set

Steps 2–3 repeat for every candidate. The test set appears only in step 5.

⚠️ **Never tune on the test set.** If it guides your choice, it is no longer unseen, and the final performance estimate becomes too optimistic.

🔍 GridSearchCV: How It Works

Try every combination in a grid; score each with cross-validation.

How GridSearchCV works

1. Generates all hyperparameter combinations in param_grid.
2. Evaluates each combination using K-fold cross-validation.
3. Averages the scores and ranks the combinations.
4. Retrains the best model on the full training set.

Search space in the code below

6 values of λ (ridge__alpha)

× 5 CV folds = 30 fits

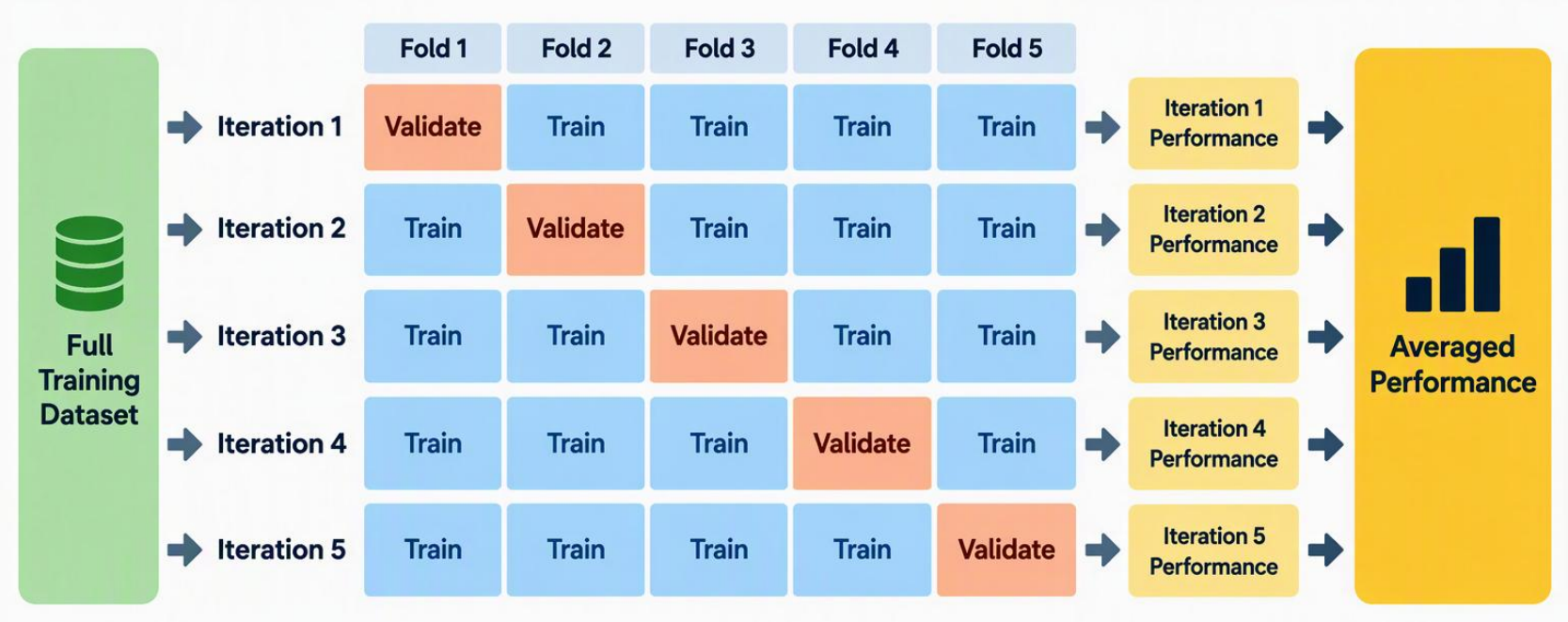
+ 1 final refit on all training data

grid.best_params_ gives the winning λ

```
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.datasets import fetch_california_housing
# Data: hold out 20% as the final test set
X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Pipeline: scaling is refit inside every CV round, so nothing leaks
pipe = make_pipeline(StandardScaler(), Ridge())
# Grid: candidate λ values; "ridge__alpha" = pipeline step name + parameter name
param_grid = {"ridge__alpha": [0.001, 0.01, 0.1, 1, 10, 100]}
# 6 values × 5 folds = 30 fits, then the best λ is refit on all training data
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="neg_root_mean_squared_error")
grid.fit(X_train, y_train)
```

💡 **Cost = combinations × folds.** Two hyperparameters with 6 values each and 5 folds mean 180 fits.

K-Fold Cross-Validation



How K-fold works

1. Split the training data into K equal folds
2. Hold out one fold for validation
3. Train on the other K−1 folds and score
4. Repeat K times, so each fold validates once
5. Average the K scores: a more stable estimate than a single split

💡 **Choosing K:** 5 or 10 is typical. Larger K means less bias but more fits (K per candidate).

🧰 Key Parameters and Outputs

Configure the search with a few parameters, then read four results.

Parameter	What it does	Example
`estimator`	Model or pipeline to tune	`make_pipeline(StandardScaler(), Ridge())`
`param_grid`	Parameter name → values to try	`{"ridge__alpha": [0.001, 0.01, ... 100]}`
`cv`	Number of cross-validation folds	`cv=5`
`scoring`	Metric used to rank (higher = better)	`"neg_root_mean_squared_error"`
`refit`	Retrain best setting on all training data	`True (default)`

```
# After grid.fit(X_train, y_train)
# (outputs below are illustrative)

print(grid.best_params_)
# {'ridge__alpha': 10}

print(-grid.best_score_)
# 0.7236  (RMSE, sign flipped)

best_model = grid.best_estimator_
# Pipeline refit on all training data

cv = pd.DataFrame(grid.cv_results_)
# One row per λ → next slide
```

Attribute	What you get
`best_params_`	Winning hyperparameter values (dict)
`best_score_`	Mean CV score of the winner
`best_estimator_`	Model refit on all training data; use .predict()
`cv_results_`	Scores of every combination; wrap in a DataFrame

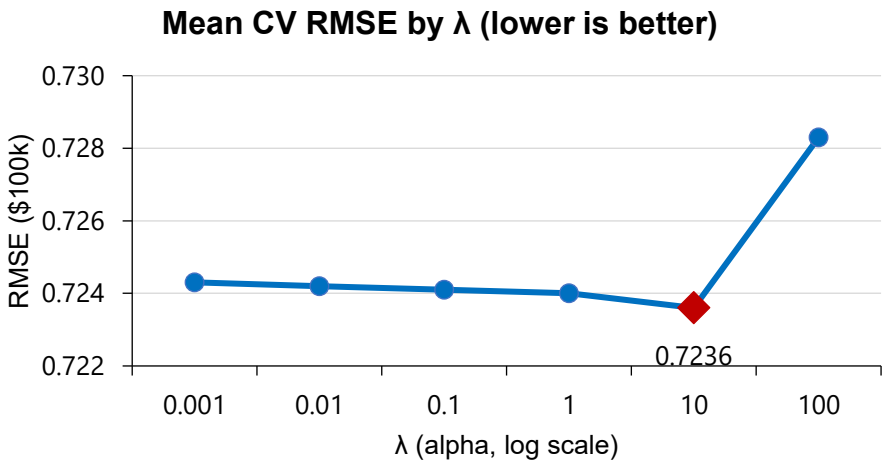
⚠️ **best_score_ is negative RMSE:** flip the sign.

📊 Analyzing the Results

Compare every λ with cv_results_, then choose the best setting.

λ (alpha)	Mean RMSE	Std	Rank
0.001	0.7243	0.0182	5
0.01	0.7242	0.0182	4
0.1	0.7241	0.0181	3
1	0.7240	0.0178	2
10	**0.7236**	**0.0170**	**1**
100	0.7283	0.0162	6

Illustrative output; exact values vary slightly by version.




```
cv = pd.DataFrame(grid.cv_results_)                # one row per λ
cols = ["param_ridge__alpha", "mean_test_score", "std_test_score", "rank_test_score"]
table = cv[cols].copy()
table["mean_test_score"] = -table["mean_test_score"]   # negative RMSE → RMSE
print(table.sort_values("rank_test_score"))             # best λ first
table.plot(x="param_ridge__alpha", y="mean_test_score", logx=True, marker="o")  # chart
```

How to interpret

✅ Best setting = rank 1 (lowest mean RMSE): λ = 10

✅ Error rises at λ = 100 (too much shrinkage). If the best λ sits on the grid edge, extend the grid

🎯 Practical Application: Ridge vs Lasso

Same data, same grid, same metric: only the model changes.

```
# Reuses X_train, X_test, y_train, y_test and the imports from slide 40
from sklearn.linear_model import Lasso
from sklearn.metrics import root_mean_squared_error
from sklearn.pipeline import Pipeline

# 1. Candidate λ values (named "alpha"); "model__" points to the pipeline step
param_grid = {"model__alpha": [0.001, 0.01, 0.1, 1, 10, 100]}

# 2. Run the same search for Ridge and for Lasso
for name, model in [("Ridge", Ridge()), ("Lasso", Lasso(max_iter=10000))]:
    pipe = Pipeline([("scale", StandardScaler()), ("model", model)])  # scale, then fit
    # Try every λ with 5-fold CV; scored by RMSE (sklearn negates it)
    grid = GridSearchCV(pipe, param_grid, cv=5, scoring="neg_root_mean_squared_error")
    grid.fit(X_train, y_train)

    # 3. Final check: score the best model on the untouched test set
    test_rmse = root_mean_squared_error(y_test, grid.predict(X_test))
    zeros = (grid.best_estimator_["model"].coef_ == 0).sum()  # features dropped
    # Prints: name, best λ, CV RMSE, test RMSE, zero weights
    print(name, grid.best_params_, round(-grid.best_score_, 4), round(test_rmse, 4), zeros)
```

Model	Best λ	CV RMSE	Test RMSE	Zero weights
Ridge	10	0.7236	0.7455	0 of 8
Lasso	0.001	0.7239	0.7458	0 of 8

Illustrative output; exact values vary slightly by version.

How to read it

- ✅ CV ≈ test: the chosen λ generalizes
- ✅ Best Lasso λ is tiny, so no feature is dropped
- ✅ Larger λ removes features but raises RMSE

Summary

1 The model

Learn $\hat{y} = w \cdot x + b$ by minimizing the cost $J(w, b)$ = MSE between predictions and true values.

2 Gradient descent

Update w and b step by step downhill; the learning rate α sets the step size.

3 Multiple features

Vector form $\hat{y} = w \cdot x + b$. Same learning process: split → scale → fit → evaluate → predict.

4 Evaluation

Use MAE, RMSE and R^2 on the test set. A good model beats a simple baseline.

5 Regularization

Cost = MSE + λ × penalty. Ridge (L2) shrinks weights; Lasso (L1) sets some to zero.

6 Hyperparameter Tuning

Select the best λ using K-fold cross-validation, refit the model on all training data, and evaluate its final performance on the test set.

Resources

- **Linear models (scikit-learn):** Ridge, Lasso and linear regression user guide
  https://scikit-learn.org/stable/modules/linear_model.html

- **Cross-validation:** K-fold and cross_val_score
  https://scikit-learn.org/stable/modules/cross_validation.html

- **Tuning hyperparameters:** GridSearchCV
  https://scikit-learn.org/stable/modules/grid_search.html

- **Course: ML Specialization** (Andrew Ng): regression and gradient descent
  https://www.deeplearning.ai/specializations/machine-learning